AI Security Findings

Overall risk: HIGH | Total findings: 10

Executive summary

Risk overview

Total findings: 10. Critical: 0, High: 9, Medium: 1, Low: 0.

Technical summary

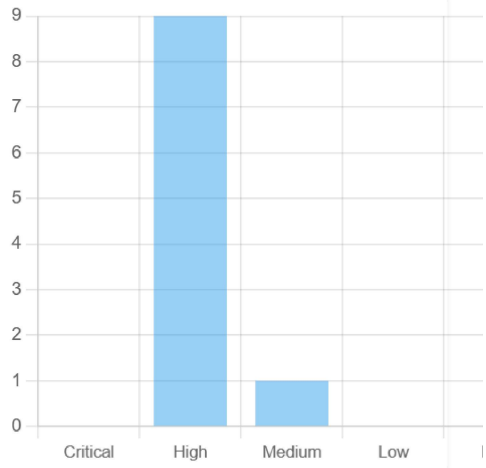
Overall Risk Level: HIGH

Key Issues to Fix:
1. Insecure Authentication: The token verification mechanism accepts tokens starting with "test-" and relies on hardcoded tokens. Replace this with a secure solution such as JWT with proper signature verification and manage secrets through a secure store.
2. Hardcoded Secrets and Credentials: Database credentials, JWT secrets, and API keys are embedded in source code and configuration files. Shift these to secure storage using environment variables or a dedicated secrets management system and ensure strict file permission controls.
3. SQL Injection Vulnerabilities: Several SQL queries directly interpolate untrusted user input without sanitization or prepared statements across multiple functions. Refactor these queries to use parameterized queries or prepared statements to prevent SQL injection attacks.
4. Command Injection: The /run route accepts unsanitized 'cmd' input, risking arbitrary shell command execution. Validate, sanitize, or whitelist allowed commands and consider using safer alternatives like the subprocess library.
5. Reflected XSS and Sensitive Data Logging: Untrusted user input is inserted in the DOM via innerHTML, leading to XSS risks, and sensitive user credentials (passwords) are logged in plain text. Utilize safer DOM manipulation methods (e.g., textContent) or sanitize input, and remove sensitive data from logs.

Quick Wins and Hardening Recommendations:
• Immediately remove or obfuscate all hardcoded tokens, API keys, and database credentials.
• Refactor all SQL queries and shell command executions to use secure, parameterized interfaces.
• Update the authentication mechanism to use industry-standard solutions.
• Review logging practices to ensure that sensitive information is not recorded.
• Conduct a comprehensive audit of client-side code to sanitize user input and prevent XSS.

These improvements are critical to mitigating the identified vulnerabilities and enhancing the overall security posture of the project.

Severity distribution



Severity	Title	File / Location	Description	Recommendation
HIGH	Weak Token Verification and Hardcoded Tokens	`verify_token_insecure function`	The function uses an insecure authentication mechanism where tokens are verified against a	Replace the current token verification with a robust solution. Use secure token generation methods (e.g., JWT with proper signature verification) and store secrets securely (e.g., in environment variables or a secrets manager). Remove the fallback logic that allows any token

Severity	Title	File / Location	Description	Recommendation
			hardcoded list and any token starting with 'test-' is accepted. Hardcoding tokens and allowing such predictable patterns can be easily bypassed by attackers, leading to unauthorized access.	starting with 'test-' and avoid using hardcoded tokens.
HIGH	Hardcoded Database Credentials	`Insecure configuration file`	The configuration file contains hardcoded database credentials (DB_HOST, DB_USER, DB_PASSWORD). This vulnerability could allow attackers to gain unauthorized access to the database if the configuration file is exposed.	Store sensitive credentials in a secure location, such as environment variables or a secrets management system. Ensure restricted file permissions for configuration files containing sensitive data.
HIGH	Hardcoded JWT Secret	`Insecure configuration file`	The JWT_SECRET is hardcoded in the configuration file. If this file is compromised, attackers could forge JWT tokens and bypass authentication mechanisms.	Remove hardcoded secrets from configuration files. Use environment variables or a dedicated secrets management tool to securely manage sensitive keys and secrets.
HIGH	SQL Injection	`get_user function, line with query string concatenation`	The user-supplied input 'uid' is directly concatenated into the SQL query string without any sanitization or parameterization, which exposes the application to SQL Injection vulnerabilities.	Use parameterized queries or prepared statements to safely include user input. For example, the query could be rewritten as: 'SELECT * FROM users WHERE id = ?' with the parameter (uid,) passed to the execute method.
HIGH	SQL Injection Vulnerability	`get_user_raw_query, line where query is constructed`	The code directly concatenates the untrusted user_id into the SQL query string. This allows an attacker to	Use parameterized queries with placeholders to safely handle user input. For example, use 'cur.execute("SELECT id, username, email FROM users WHERE id = ?", (user_id,))' to prevent SQL injection.

Severity	Title	File / Location	Description	Recommendation
			manipulate the query and perform SQL injection attacks.	
MEDIUM	Hardcoded Secret	`API_KEY variable`	The API key is hardcoded in the source code, making it susceptible to exposure and misuse if the source code is leaked or accessed by unauthorized parties.	Store secrets in a secure environment variable or secret management tool, not directly in source code, and ensure access is properly controlled.
HIGH	Command Injection	`/run route`	User input from the 'cmd' parameter is passed directly to os.popen() without sanitization, allowing an attacker to execute arbitrary shell commands on the server.	Avoid executing shell commands with unsanitized user input. Use safer alternatives, validate and sanitize input, or use whitelisting for allowed commands. Consider using subprocess with a properly defined command list if execution is necessary.
HIGH	SQL Injection	`/user route`	User input from the 'id' parameter is concatenated into an SQL query string without sanitization or parameterization. This can allow attackers to manipulate the query to execute arbitrary SQL code.	Use parameterized queries or prepared statements to safely include user input in SQL queries. Validate and sanitize user input, and consider using an ORM framework for database operations to mitigate SQL injection risks.
HIGH	Reflected Cross-Site Scripting (XSS) Vulnerability	`document.getElementById('welcome').innerHTML assignment`	The code directly inserts untrusted user input from the URL query parameter 'name' into the DOM using innerHTML. This can allow an attacker to inject and execute arbitrary JavaScript code, resulting in a reflected XSS vulnerability.	Avoid inserting untrusted data into the DOM using innerHTML. Use textContent or DOM manipulation methods that do not parse HTML, e.g., document.getElementById('welcome').textContent = 'Hello, ' + name + '!'. Alternatively, sanitize the input to remove malicious content before inserting it into the HTML.
HIGH	Sensitive Data	`log_login_attempt function`	The code logs the user's	Avoid logging sensitive information such as passwords. If you need to log data about the

Severity	Title	File / Location	Description	Recommendation
	Logging		password in plain text when a login attempt is made. This may lead to exposure of sensitive credentials in the logs.	login attempt, log minimal information (e.g., username and the fact that a login attempt occurred) without including passwords. Additionally, restrict access to logs to authorized personnel only.